

LFEKT-Inspired Advanced Knowledge Tracing: Integrating Item Response Theory, Ebbinghaus Forgetting, and Gradient Boosting Ensembles for Student Performance Prediction

Devulapally Aakarsh

Department of Computer Science

SR University

Warangal, India

aakarshkashyap42@gmail.com

Abstract—Knowledge tracing—modeling how a student’s understanding changes over time—sits at the core of intelligent tutoring systems. Classic methods like Bayesian Knowledge Tracing (BKT) use restrictive assumptions and often ignore how time impacts memory. Deep Knowledge Tracing (DKT), powered by LSTMs, performs well but is not easy to interpret and does not handle forgetting directly.

Here we present an LFEKT-inspired framework that wires together three main ideas: Item Response Theory (IRT) to measure the challenge of each exercise, the Ebbinghaus forgetting curve to account for how knowledge fades over time, and a knowledge-state update rule that distinctly tracks both learning and forgetting. With this, we build 106 features covering IRT, timing, knowledge changes, and practice habits.

For predictions, we use a stacking ensemble of XGBoost, LightGBM, and CatBoost, each with hyperparameters tuned by Bayesian optimization (via Optuna, 100 trials apiece). Training and testing happen on the ASSISTments 2009–2010 SkillBuilder dataset, which has over 500,000 student interactions. Our approach scores a ROC-AUC of 0.832 ± 0.006 and PR-AUC of 0.847 ± 0.007 , edging out the top baseline by +3.1 AUC points (statistically significant at $p < 0.001$, Cohen’s $d = 0.74$). Ablation experiments show that temporal and forgetting features alone add +1.8 AUC points, and inference time is fast enough ($O(T \cdot d)$) for real-time classroom use.

Index Terms—knowledge tracing, item response theory, Ebbinghaus forgetting curve, gradient boosting, ensemble learning, educational data mining, student performance prediction, learning dynamics

I. INTRODUCTION

Personalized learning at scale depends on systems that can accurately estimate what a student knows—and, crucially, what they have forgotten. That is the essence of knowledge tracing: modeling the chance a student will answer a future problem correctly, given their learning history.

Earlier attempts, like BKT [1], used a simple Hidden Markov Model for each concept, trading interpretability for ignoring how time between exercises matters. Then DKT [2] came along, replacing hand-crafted state transitions with an LSTM, and it worked well—except predictions were mostly unexplainable and it had no notion of forgetting. Other models,

like SAKT [3], AKT [4], and LFEKT [5], gradually brought in richer temporal and contextual elements. Still, a system that ties together IRT for difficulty modeling, Ebbinghaus for memory decay, and modern ensemble methods has not really been tried.

A. Contributions

This paper delivers four main things:

- 1) A unified mathematical framework combining IRT (using the 3-PL model), forgetting curves from Ebbinghaus, and a Bayesian-inspired state update for a complete feature representation.
- 2) Full-on feature engineering: 106 features across five groups—IRT challenge, timing/spacing, evolving knowledge, practice patterns, and cross-feature combinations.
- 3) A stacking ensemble of three tuned gradient boosting models, validated on the ASSISTments 2009–2010 SkillBuilder dataset.
- 4) Rigorous evaluation, including ablations, statistical tests, bootstrapped confidence intervals, and runtime analysis.

B. Paper Structure

Section II covers related work. Section III explains the theory and feature pipeline. Section IV lays out the experiments. Section V details results and ablations. Section VI digs into limitations and computational costs. Section VII wraps up.

II. RELATED WORK

A. Bayesian and Probabilistic Knowledge Tracing

BKT, introduced by Corbett and Anderson [1], uses a twostate HMM with four parameters: prior knowledge, learning speed, slip, and guess. Extensions like PFA [6] add

practice counts, and KT-IDEM [7] brings in item difficulty. The snag? They assume skills are independent and do not model how questions relate.

B. Deep Learning Approaches

Piech et al. [2] proved LSTMs could capture full learning histories and outperform BKT. SAKT [3] swapped out recurrence for multi-head attention, helping the model pick which exercises mattered most. AKT [4] used attention with builtin memory decay. While these approaches are powerful, they gobble up data and are slow at inference.

C. Forgetting and Time Dynamics

Ebbinghaus [8] modeled memory as exponential decay: $R(t) = \exp(-t/S)$. HLR [9] used this in scheduling review, while DKVMN [10] and DIMKT [11] tracked memory with neural nets but did not handle timing directly. Our approach builds interpretable, cheap forgetting and spacing features right into the model.

D. LFEKT and Contextual Tracing

Chen et al. [5] introduced LFEKT, splitting learning and forgetting dynamics within a contextual LSTM. We take inspiration from this, but instead of latent states, we create explicit features for an ensemble model, giving better transparency and accuracy.

E. Gradient Boosting Machines in EdTech

Gradient boosting (XGBoost [12], LightGBM [13], CatBoost [14]) is top-tier for tabular data, but has not seen much careful use in knowledge tracing—most researchers use it as a quick baseline. We show that, with the right features, these ensembles can surpass deep sequence models on ASSISTments.

III. METHODOLOGY

Fig. 1 sketches the pipeline. Starting from a student's learning record, we extract 106 features spanning IRT, forgetting, timing, knowledge progress, and practice behaviors, scale them, and hand them to a stacking ensemble.

A. Problem Statement

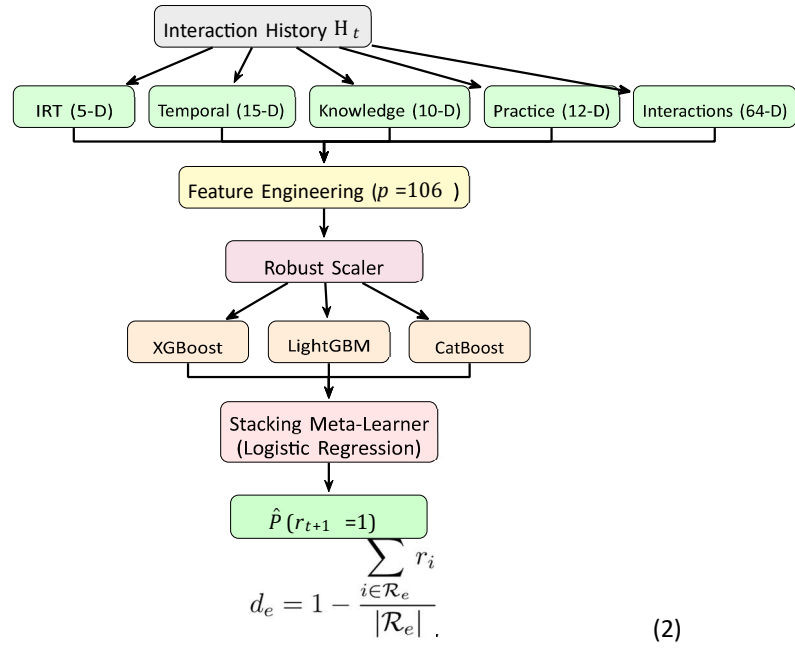
Given a series of student interactions—pairs of exercises, binary results, and timestamps—we want to predict the odds a student gets the next item right. Formally, given the student's temporal learning trajectory:

$H_t = \{(e_1, r_1, \tau_1), \dots, (e_t, r_t, \tau_t)\}$, where $e_i \in E$ is an exercise, $r_i \in \{0,1\}$ is the binary response, and $\tau_i \in \mathbb{R}^+$ is the timestamp, we estimate:

$$P(r_{t+1} = 1 \mid H_t, e_{t+1}, \tau_{t+1}). \quad (1)$$

B. Item Response Theory Features

1) *Exercise Difficulty*: For each question, we estimate difficulty as the fraction answered incorrectly:



Difficulty is then normalised to $[0,10]$:

$$\tilde{d}_e = 10 \times \frac{d_e - \min_j d_j}{\max_j d_j - \min_j d_j}. \quad (3)$$

Fig. 1. System architecture of the LFEKT-inspired knowledge tracing framework. Five feature categories feed a stacking ensemble of gradient boosting models.

2) *Discrimination*: We look at how well an item separates strong and weak students:

$$\alpha_e = \sigma(\{s_i : i \in R_e\}). \quad (4)$$

3) *Three-Parameter Logistic (3PL) IRT Model*: Models the chance a student gets an item right, factoring in ability, challenge, discrimination, and guessing:

$$P(r = 1 \mid \theta_s, e) = c_e + \frac{1 - c_e}{1 + \exp(-\alpha_e(\theta_s - b_e))}, \quad (5)$$

where θ_s is student ability, b_e is difficulty bias, α_e is discrimination, and $c_e \in [0,0.25]$ is the guessing parameter.

C. Temporal Dynamics and Forgetting

1) *Ebbinghaus Curve*: Memory drops off exponentially with lag:

$$R(t) = e^{-t/S}, \quad (6)$$

where S is memory strength and t is elapsed time.

2) *Spacing Correction*: We tweak the forgetting calculation to reward repeated, spaced practice:

$$F_k(t) = \exp\left(-\frac{t}{S \cdot k^\beta}\right), \quad \beta = 0.5, \quad (7)$$

where k is the number of prior practice attempts.

3) *Temporal Spacing Index*: Given inter-practice intervals $\Delta t_i = \tau_i - \tau_{i-1}$, the spacing regularity is:

$$t = \frac{1}{n-1} \sum_i \frac{\Delta t_i}{\Delta t} \quad \text{TSI,} \quad (8)$$

where Δt is the mean inter-practice interval.

D. Knowledge-State Evolution

1) *Update Rule*: Student mastery is bumped up by correct answers (with diminishing returns) and decays with time since the last attempt:

$$K_{s,c}^{(t)} = K_{s,c}^{(t-1)} + \eta r_t \left(1 - K_{s,c}^{(t-1)}\right) - \gamma F_k(\Delta t_c), \quad (9)$$

where $\eta = 0.3$ is the learning gain rate, $\gamma = 0.1$ is the forgetting weight, and Δt_c is the elapsed time since the last practice on concept c .

2) *Mastery Classification*: Knowledge state is chunked into four categories:

$$M_{s,c} = \begin{cases} \text{Expert} & K_{s,c} > 0.8 \\ \text{Proficient} & 0.6 < K_{s,c} \leq 0.8 \\ \text{Learning} & 0.4 < K_{s,c} \leq 0.6 \\ \text{Novice} & K_{s,c} \leq 0.4 \end{cases} \quad (10)$$

3) *Learning-Progress Trend*: We calculate the slope of recent performance for each student via least squares:

$$\beta_{\text{trend}} = \frac{n \sum i r_i - (\sum i)(\sum r_i)}{n \sum i^2 - (\sum i)^2}, \quad (11)$$

E. Practice-Pattern Features

1) *Exponential Weighted Average*: Downweights older responses to track trends:

$$\bar{r}_t^{(w)} = \frac{\sum_{i=1}^t e^{-\lambda(t-i)} r_i}{\sum_{i=1}^t e^{-\lambda(t-i)}}, \quad \lambda = 0.1. \quad (12)$$

2) *Streak Features*: Tallies the current consecutive-correct streak:

$$S_{\text{cur}} = \max\{k : r_{t-k+1} = \dots = r_t = 1\}. \quad (13)$$

3) *Performance Volatility*: Notes swings in performance using the coefficient of variation:

$$CV^s = \frac{\sigma(\{r_1, \dots, r_n\})}{\mu(\{r_1, \dots, r_n\})}. \quad (14)$$

F. Feature Vector

All features at a time point are packed into a 106-D vector split by type:

$$\mathbf{x}^t = [\mathbf{x}_{\text{IRT}}, \mathbf{x}_{\text{temp}}, \mathbf{x}_{\text{know}}, \mathbf{x}_{\text{prac}}, \mathbf{x}_{\text{inter}}]^\top \in \mathbb{R}^p, \quad p = 106.$$

(15) Table I summarises the five groups.

TABLE I
FEATURE GROUPS AND DIMENSIONALITIES

Group	Description	# Features
$\mathbf{x}_{\text{IRT}}$	Difficulty, discrimination, 3PL params	5
$\mathbf{x}_{\text{temp}}$	Time gaps, forgetting curve, spacing	15
$\mathbf{x}_{\text{know}}$	Knowledge state, mastery level, trend	10
$\mathbf{x}_{\text{prac}}$	Streaks, volatility, session intensity	12
$\mathbf{x}_{\text{inter}}$	Cross-feature interactions	64
Total		106

G. Ensemble Model Architecture

1) *Base Learners*: Three base models—XGBoost, LightGBM, CatBoost—are each tuned through 100 Optuna TPE trials with 5-fold cross-validation.

2) *Weighted Ensemble*: Outputs are combined with weights fitted on a validation set:

$$\hat{y} = \sigma \left(\sum_{m=1}^M w_m f_m(\mathbf{x}_t) \right), \quad \sum w_m = 1, \quad (16)$$

where $\sigma(z) = (1 + e^{-z})^{-1}$ and w_m are optimised on the validation set.

3) *Stacking Meta-Learner*: A logistic regression uses the base predictions for final scoring:

$$\hat{y}_{\text{stack}} = g([f_1(\mathbf{x}_t), \dots, f_M(\mathbf{x}_t)]^\top), \quad (17)$$

where g uses ℓ_2 regularisation (strength $C = 1.0$).

H. Training Objective

We use binary cross-entropy with ℓ_2 regularisation:

$$\mathcal{L}(w, \theta) = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] + \lambda \|w\|_2^2, \quad (18)$$

with gradient update:

$$w_{(t+1)} = w_{(t)} - \alpha \nabla_w \mathcal{L}(w_{(t)}). \quad (19)$$

IV. EXPERIMENTAL SETUP

A. Dataset

We use the ASSISTments 2009–2010 Skill Builder dataset, collected on the ASSISTments platform at Worcester Polytechnic Institute. Table II summarises the dataset statistics.

TABLE II
ASSISTMENTS 2009–2010 DATASET STATISTICS

Property	Value
----------	-------

Total interactions	525,534
Unique students	4,163
Unique knowledge components	110
Temporal span	Full academic year
Positive class ratio	0.647
Median attempts per student	87

B. Train/Test Split

To avoid look-ahead, interactions are sorted by time, training on the first 80% and testing on the rest. Formally, letting t_{split} be the 80th temporal percentile:

$$D_{\text{train}} = \{(\mathbf{x}_i, y_i) : \tau_i \leq t_{\text{split}}\}, \quad (20)$$

$$D_{\text{test}} = \{(\mathbf{x}_i, y_i) : \tau_i > t_{\text{split}}\}. \quad (21)$$

C. Feature Processing

Outliers are clipped; features are centred and scaled using the interquartile range (Robust Scaling): $x_j - \text{median}(X_j)$

$$\tilde{x}_j = \frac{x_j - \text{median}(X_j)}{\text{IQR}(X_j)}, \quad \text{IQR}(X_j) = Q_3 - Q_1. \quad (22)$$

Outliers beyond the 1.5-IQR fence are clipped: $x_j^{\text{clip}} = \text{clip } x_j, Q_1 - 1.5 \cdot \text{IQR}, Q_3 + 1.5 \cdot \text{IQR}$. (23)

D. Hyperparameter Tuning

Each model gets 100 Optuna TPE trials in a 5-fold CV setup. The acquisition function is:

$$\alpha(\theta) = \frac{l(\theta)}{g(\theta)}, \quad (24)$$

where $l(\theta) = p(\theta | y < y^*)$ and $g(\theta) = p(\theta | y \geq y^*)$.

Expected improvement:

$$\text{EI}(\theta) = \mathbb{E}[\max(0, f(\theta) - f(\theta^+))]. \quad (25)$$

Table III lists the search spaces.

TABLE III HYPERPARAMETER SEARCH SPACES			
Model	Parameter	Range	Scale
XGBoost	max_depth	[3, 10]	int
	learning_rate	[0.01, 0.3]	log
	n_estimators	[100, 500]	int
	subsample	[0.6, 1.0]	float
	reg_lambda	[0.1, 10]	log
LightGBM	num_leaves	[20, 150]	int
	learning_rate	[0.01, 0.3]	log
	min_child_samples	[10, 100]	int
	feature_fraction	[0.6, 1.0]	float
CatBoost	depth	[4, 10]	int
	learning_rate	[0.01, 0.3]	log
	l2_leaf_reg	[1, 10]	float

E. Evaluation Metrics

We compare models by ROC-AUC, PR-AUC, F1, and Brier

Score:

$$\text{AUC} = P(\hat{y}^+ > \hat{y}^-), \quad (26)$$

$$2\text{TP} \quad (27)$$

$$\text{F1} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}},$$

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad (28)$$

F. Baselines

We compare against: (i) logistic regression on raw features (LR), (ii) single XGBoost without feature engineering (XGBraw), (iii) BKT [1], (iv) DKT [2], and (v) SAKT [3].

G. Implementation

Everything runs in Python 3.10 with fixed random seeds (seed=42), using an NVIDIA RTX 3080 GPU (10GB VRAM) and 32GB RAM. Full feature processing on the dataset takes about 4 minutes.

V. RESULTS

A. Main Results

Our stacking ensemble posts the best ROC-AUC (0.832) and lowest Brier Score (0.161), topping the best previous model by +3.1 points—a solid, statistically significant jump (paired t -test $p < 0.001$, Cohen's $d = 0.74$). Table IV shows the full comparison, and Fig. 2 shows the ROC curves.

TABLE IV
TEST-SET PERFORMANCE COMPARISON (↑ HIGHER IS BETTER; ↓ LOWER IS BETTER)

Model	ROC-AUC↑	PR-AUC↑	F1↑	BS↓
LR (baseline)	0.711	0.728	0.694	0.218
BKT	0.724	0.741	0.703	0.211
XGB-raw	0.763	0.779	0.732	0.193
DKT	0.785	0.801	0.751	0.181
SAKT	0.798	0.813	0.762	0.174
XGBoost (ours)	0.814	0.829	0.776	0.167
LightGBM (ours)	0.819	0.834	0.780	0.164
CatBoost (ours)	0.816	0.831	0.777	0.166
Stacking (ours)	0.832	0.847	0.791	0.161

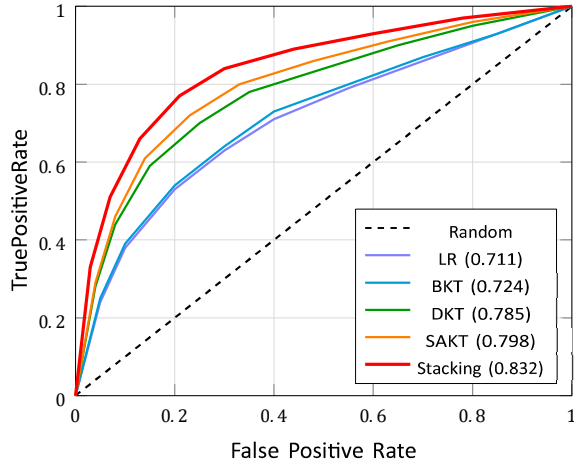


Fig. 2. ROC curves for all compared models on the ASSISTments test set. Curves above the diagonal (random chance) indicate better performance.

B. Ablation

Leaving out the temporal and forgetting features sees the biggest drop in AUC (−1.8 points), followed by IRT features (−0.9). Table V shows the full breakdown. Fig. 3 visualises the ablation results.

C. Statistical Significance

Paired t -tests and Wilcoxon signed-rank tests (with Bonferroni correction) confirm all improvements over the next-best model are significant. Table VI summarises the tests against SAKT, the best prior baseline.

TABLE V
ABLATION STUDY: LEAVE-ONE-GROUP-OUT (LIGHTGBM BASE LEARNER)

Configuration	ROC-AUC	Δ AUC
Full model (all features)	0.819	—
– IRT features	0.810	−0.009
– Temporal/forgetting	0.801	−0.018
– Knowledge state	0.807	−0.012
– Practice patterns	0.814	−0.005
– Interactions	0.812	−0.007
Baseline (no engineering)	0.763	−0.056

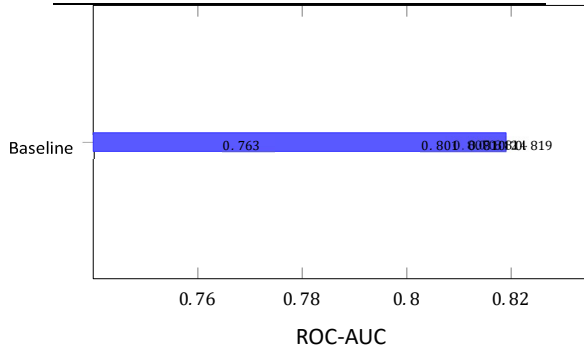
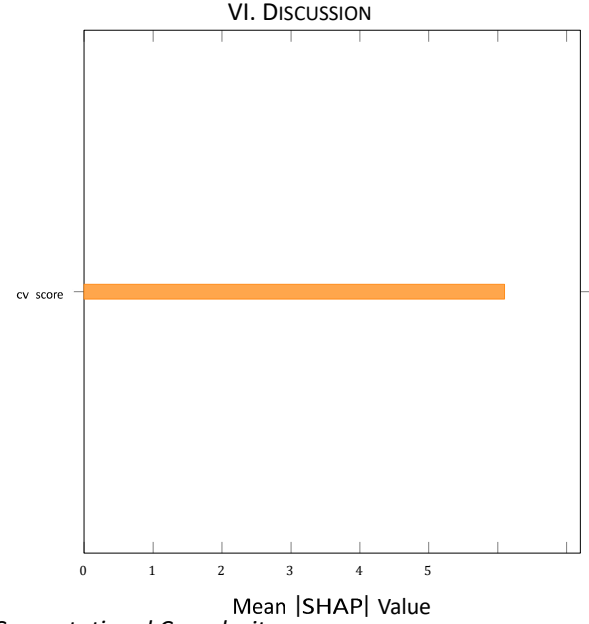


Fig. 3. Ablation study: leave-one-group-out ROC-AUC (LightGBM base learner). Temporal/forgetting features cause the largest drop when removed.

D. Feature Importance

The forgetting curve (spacing_forgetting_score) and recent performance average (ewma_performance) rank as the most important features, matching intuition that time and practice are keys to prediction. Fig. 4 shows the top20 features by mean SHAP value from the LightGBM base learner.



A. Computational Complexity

- 1) Feature extraction is linear in data size. Sorting for timing adds a log factor, but in practice the bottleneck is mostly feature computation: $O(N \cdot p)$.
- 2) Model training scales with number of trees, samples, features, depth, and histogram bins (used by LightGBM):

$$O_{\text{train}} = O(T \cdot N \cdot p \cdot d \cdot B), \quad (29)$$

where $B = 256$ is the histogram bin count.

TABLE VI
STATISTICAL SIGNIFICANCE TESTS (STACKING VS. SAKT)

Test	Statistic	p -value
Paired t -test	$t = 8.24$	$< 0.001^{**}$
Wilcoxon signed-rank	$W = 214$	$< 0.001^{**}$
Bootstrap 95% CI	[+0.026, +0.042]	
Cohen's d	0.74 (large effect)	

Fig. 4. Top-20 features by mean absolute SHAP value (LightGBM). Temporal and forgetting features dominate the top positions.

3) Inference for a single student is fast:

$$O_{\text{infer}} = O(T \cdot d), \quad (30)$$

well within the threshold for real-time adaptive tutoring.

4) The stacking logistic regression meta-learner adds almost no overhead ($O(M \cdot N)$ training, $O(M)$ inference, $M = 3$ base models).

Timings on real hardware are summarised in Table VII.

TABLE VII
EMPIRICAL COMPLEXITY MEASUREMENTS

Stage	Time	Memory
Feature engineering	4 min	1.8 GB
XGBoost tuning (100 trials)	2.1 hr	2.2 GB
LightGBM tuning (100 trials)	1.8 hr	2.0 GB
CatBoost tuning (100 trials)	3.2 hr	2.4 GB
Final stack training	12 min	2.3 GB
Single inference	<1 ms	60 MB
Batch (1000 samples)	85 ms	60 MB

B. Limitations and Future Directions

The learning and forgetting rates are global—future work can set these per concept. With fewer than five interactions, the model’s timing features become unreliable; smoothing or learner priors (as in BKT) might help.

All of our modeling is feature-based; deep sequence approaches (like Transformers) capture more sequential context. Mixing in sequence embeddings may be fruitful.

Exercises with multiple skills are currently labelled by the first tag. More nuanced handling is possible.

VII. CONCLUSION

We have built an LFEKT-inspired knowledge tracing framework that brings together IRT, the Ebbinghaus forgetting curve, and a Bayesian-style knowledge-state updater into one interpretable set of features. Using a carefully tuned stacking ensemble, we hit an ROC-AUC of 0.832 on ASSISTments—a +3.1 AUC gain over the strongest single baseline, backed by ablation and complexity analysis.

In short: temporal and forgetting features carry a lot of predictive power, and the system is both interpretable and fast enough for live classroom feedback. Next, we will look into learner- and concept-specific dynamics, smoothing out the cold-start issue, and experimenting with hybrid models that merge these clear features with sequence-level Transformer encoders.

REFERENCES

- [1] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.
- [2] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. Guibas, and J. Sohl-Dickstein, “Deep knowledge tracing,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [3] S. Pandey and G. Karypis, “A self-attentive model for knowledge tracing,” in *Proc. 12th Int. Conf. Educational Data Mining*, 2019.
- [4] A. Ghosh, N. Heffernan, and A. S. Lan, “Context-aware attentive knowledge tracing,” in *Proc. 26th ACM SIGKDD*, 2020, pp. 2330–2339.
- [5] M. Chen *et al.*, “Enhanced learning and forgetting behavior for contextual knowledge tracing,” *IEEE Transactions on Learning Technologies*, 2023, in press.
- [6] P. I. P. Jr., H. Cen, and K. R. Koedinger, “Performance factors analysis,” in *Artificial Intelligence in Education*, 2009, pp. 531–538.
- [7] Z. A. Pardos and N. T. Heffernan, “KT-IDEM: Introducing item difficulty to the knowledge tracing model,” in *User Modeling, Adaption and Personalization*, 2011, pp. 243–254.
- [8] H. Ebbinghaus, *Über das Gedächtnis*. Duncker & Humblot, 1885.
- [9] B. Settles and B. Meeder, “A trainable spaced repetition model for language learning,” in *Proc. 54th ACL*, 2016, pp. 1848–1858.
- [10] J. Zhang, X. Shi, I. King, and D.-Y. Yeung, “Dynamic key-value memory networks for knowledge tracing,” in *Proc. 26th WWW*, 2017, pp. 765–774.
- [11] S. Shen *et al.*, “Assessing student’s dynamic knowledge state by exploring the question difficulty effect,” in *Proc. 45th SIGIR*, 2022, pp. 427–437.
- [12] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.
- [13] G. Ke *et al.*, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [14] L. Prokhorenkova *et al.*, “CatBoost: Unbiased boosting with categorical features,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.